

RAPPORT DE MAINTENANCE CORRECTIVE

Systeme de Réserveation de Ressources (SRR)

Tech-Academy

BTS SIO - Option SLAM - Épreuve E6 - Session 2026

Cluzet Emma

1. Objet du document

Ce rapport rend compte de la phase de maintenance corrective du projet SRR. Il recense les anomalies rencontrées lors de la mise en service et des premiers tests de l'application, puis décrit pour chacune le symptôme observé, la cause identifiée et la correction apportée.

Cette démarche s'inscrit dans la compétence « Assurer la maintenance corrective ou évolutive d'une solution applicative » du référentiel du BTS SIO. Elle illustre la capacité à diagnostiquer un dysfonctionnement, à en comprendre l'origine et à le corriger de façon durable.

2. Synthèse des anomalies traitées

Douze anomalies ont été identifiées et corrigées. Elles se répartissent entre des problèmes de configuration de l'environnement, des erreurs dans le code source, des problèmes liés à la base de données et un problème d'encodage.

N°	Anomalie	Type	Statut
A1	JDK 21 introuvable par l'éditeur de code	Configuration	Corrigée
A2	Composants JavaFX manquants au lancement	Configuration	Corrigée
A3	Erreur de compilation dans MainApp (méthode ifPresent)	Bug code	Corrigée
A4	Commande psql non reconnue	Environnement	Corrigée
A5	Échec d'authentification : mot de passe BCrypt invalide	Bug données	Corrigée
A6	Connexion refusée : service PostgreSQL arrêté	Environnement	Corrigée
A7	Table inexistante au lancement de l'application	Base de données	Corrigée
A8	Plantage lors d'une réservation sans champ rempli	Bug code	Corrigée
A9	Format d'heure refusé lors d'une réservation	Bug code	Corrigée
A10	Liste déroulante affichant une référence d'objet	Bug code	Corrigée
A11	Pilote JDBC PostgreSQL introuvable	Configuration	Corrigée
A12	Caractères accentués mal affichés	Encodage	Corrigée

3. Détail des anomalies et de leur correction

A1. JDK 21 introuvable par l'éditeur de code

Symptôme observé

Au démarrage, l'éditeur affichait le message « The path points to a missing or inaccessible folder » concernant la variable `java.jdt.ls.java.home`, empêchant l'analyse du code Java.

Cause identifiée

Le chemin du JDK déclaré dans la configuration pointait vers un dossier inexistant. Le poste comportait plusieurs versions de Java (8, 17 et 21) et le chemin n'avait pas été mis à jour avec l'emplacement réel du JDK 21.

Code ou commande à l'origine du problème :

```
C:\Program Files\Eclipse Adoptium\jdk-21.0.x
```

Correction apportée

Le chemin exact du JDK 21 installé a été récupéré, puis renseigné dans la configuration de l'éditeur, accompagné de la déclaration du runtime JavaSE-21. Le cache du serveur de langage a ensuite été réinitialisé.

Correction :

```
"java.jdt.ls.java.home": "C:\\Program Files\\Eclipse Adoptium\\jdk-21.0.11.10-hotspot"
```

A2. Composants JavaFX manquants au lancement

Symptôme observé

L'exécution directe de l'application levait l'erreur « des composants d'exécution JavaFX obligatoires pour exécuter cette application sont manquants ».

Cause identifiée

Depuis Java 11, JavaFX n'est plus inclus dans le JDK. Lancer l'application avec la commande `java` seule ne charge donc pas les modules JavaFX nécessaires.

Correction apportée

L'application doit être lancée via le plugin Maven dédié, qui télécharge et inclut automatiquement les modules JavaFX déclarés dans les dépendances.

Correction :

```
mvn clean javafx:run
```

A3. Erreur de compilation dans MainApp

Symptôme observé

Le code ne compilait pas. L'éditeur signalait : « The method `ifPresent(...)` is undefined for the type `DatabaseConnection` ».

Cause identifiée

La méthode `stop()` appelait `ifPresent()` sur l'objet `DatabaseConnection` comme s'il s'agissait d'un `Optional`. Or `getInstance()` retourne directement une instance de `DatabaseConnection` : la méthode `ifPresent()` n'existe pas pour ce type.

Code ou commande à l'origine du problème :

```
DatabaseConnection.getInstance().ifPresent(conn -> { ... });
```

Correction apportée

L'appel erroné a été remplacé par un appel direct à la méthode `closeConnection()`, encadré par un bloc try-catch pour gérer une éventuelle exception lors de la fermeture.

Correction :

```
DatabaseConnection.getInstance().closeConnection();
```

A4. Commande psql non reconnue

Symptôme observé

Dans le terminal, l'exécution des scripts SQL échouait avec le message « psql n'est pas reconnu comme commande ».

Cause identifiée

Le dossier bin de PostgreSQL, qui contient l'outil psql, n'était pas déclaré dans la variable d'environnement PATH du système.

Correction apportée

Le chemin du dossier bin de PostgreSQL a été ajouté au PATH. Les scripts ont également pu être exécutés directement depuis l'interface graphique pgAdmin.

Correction :

```
C:\Program Files\PostgreSQL\17\bin
```

A5. Échec d'authentification : mot de passe BCrypt invalide

Symptôme observé

Malgré la saisie des bons identifiants, l'interface affichait systématiquement « identifiants de connexion invalides ».

Cause identifiée

Les mots de passe hachés présents dans le jeu de données de test ne correspondaient pas réellement au mot de passe attendu. De plus, la bibliothèque jBCrypt 0.4 ne reconnaît que les empreintes au format \$2a\$; une empreinte au format \$2b\$ aurait provoqué une erreur.

Correction apportée

De nouvelles empreintes BCrypt ont été générées au format \$2a\$ pour le mot de passe attendu, puis insérées dans la table utilisateur. La méthode de création de compte utilise déjà `gensalt(10)`, qui produit bien des empreintes \$2a\$.

Correction :

```
BCrypt.hashpw(motDePasse, BCrypt.gensalt(10)); // produit une empreinte $2a$
```

A6. Connexion refusée : service PostgreSQL arrêté

Symptôme observé

Au lancement, l'application affichait une erreur de connexion à la base de données du type « Connection refused », alors que le mot de passe était correct.

Cause identifiée

Le service PostgreSQL n'était pas démarré sur le poste. Sans serveur de base de données actif, aucune connexion JDBC ne peut aboutir.

Correction apportée

Le service PostgreSQL a été démarré depuis le gestionnaire de services Windows, ou via la commande suivante dans un terminal administrateur.

Correction :

```
net start postgresql-x64-17
```

A7. Table inexistante au lancement

Symptôme observé

À la première utilisation, l'application levait une erreur SQL indiquant qu'une relation (table) n'existait pas, par exemple « relation "utilisateur" does not exist ».

Cause identifiée

L'application avait été lancée avant l'exécution du script schema.sql : la base de données existait mais ne contenait aucune table.

Correction apportée

Les scripts d'initialisation ont été exécutés dans le bon ordre avant le premier lancement : d'abord schema.sql pour créer les tables, puis data.sql pour insérer les données de test.

Correction :

```
schema.sql → data.sql → mvn clean javafx:run
```

A8. Plantage lors d'une réservation sans champ rempli

Symptôme observé

Lorsqu'on cliquait sur « Réserver » sans avoir choisi de ressource ou de date, l'application plantait avec une NullPointerException.

Cause identifiée

Le code construisait la réservation à partir des champs du formulaire sans vérifier au préalable qu'ils étaient renseignés. Un champ vide renvoyait la valeur null, provoquant l'erreur.

Correction apportée

Un contrôle de saisie a été ajouté en début de méthode : si un champ obligatoire est vide, un message d'erreur est affiché et le traitement est interrompu avant toute opération.

Correction :

```
if (ressource == null || dateD == null) { messageLabel.setText("Veuillez remplir tous les champs."); return; }
```

A9. Format d'heure refusé lors d'une réservation

Symptôme observé

Une réservation échouait lorsque l'utilisateur saisisait une heure sous la forme « 9h » ou « 9h00 » au lieu de « 09:00 ».

Cause identifiée

La conversion de la saisie en heure utilise `LocalTime.parse()`, qui attend obligatoirement le format HH:mm. Tout autre format provoque une exception.

Correction apportée

La conversion a été encadrée par un bloc try-catch affichant un message clair, et le champ de saisie indique désormais le format attendu (HH:mm) grâce à un texte d'aide.

Correction :

```
promptText="Heure début (HH:mm)"
```

A10. Liste déroulante affichant une référence d'objet

Symptôme observé

La liste déroulante des ressources affichait des libellés illisibles du type « Ressource@1a2b3c » au lieu du nom de la ressource.

Cause identifiée

Sans redéfinition de la méthode `toString()`, JavaFX affiche l'adresse mémoire de l'objet plutôt qu'un texte lisible.

Correction apportée

La méthode `toString()` a été redéfinie dans la classe `Ressource` pour retourner le nom et le type de la ressource.

Correction :

```
public String toString() { return nom + " (" + type + ")"; }
```

A11. Pilote JDBC PostgreSQL introuvable

Symptôme observé

L'application levait l'erreur « No suitable driver found for jdbc:postgresql ».

Cause identifiée

La dépendance du pilote JDBC PostgreSQL n'était pas (ou plus) présente dans le fichier `pom.xml`, ou n'avait pas été téléchargée par Maven.

Correction apportée

La dépendance `org.postgresql` a été ajoutée au `pom.xml`, puis les dépendances ont été retéléchargées avec Maven.

Correction :

```
<dependency> <groupId>org.postgresql</groupId> <artifactId>postgresql</artifactId>
</dependency>
```

A12. Caractères accentués mal affichés

Symptôme observé

Certains messages contenant des accents s'affichaient avec des caractères incorrects (par exemple « réservée » devenait « r?serv?e »).

Cause identifiée

Les fichiers du projet n'étaient pas tous encodés en UTF-8, ce qui provoquait une mauvaise interprétation des caractères accentués.

Correction apportée

L'encodage UTF-8 a été imposé pour l'ensemble du projet via la propriété correspondante dans le pom.xml, et les fichiers sources ont été enregistrés en UTF-8.

Correction :

```
<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
```

4. Bilan et recommandations

L'ensemble des anomalies a été corrigé et l'application est désormais pleinement fonctionnelle. Ce travail met en évidence l'importance de distinguer les anomalies liées à l'environnement de développement de celles présentes dans le code source ou dans les données.

Pour limiter ce type de problème à l'avenir, plusieurs recommandations peuvent être formulées :

- Documenter précisément les versions des outils requis (JDK, PostgreSQL) dans le fichier README.
- Exécuter les scripts d'initialisation de la base dans le bon ordre avant le premier lancement.
- Vérifier que le service PostgreSQL est démarré avant de lancer l'application.
- Privilégier la création des comptes via l'application pour garantir la cohérence des empreintes BCrypt.
- Toujours lancer l'application via Maven afin d'assurer le chargement des modules JavaFX.
- Conserver une trace des anomalies, comme dans le présent rapport, pour faciliter la maintenance future.